

Lab 3: A Scaling Study, and Your Machine's Roofline

Produce the two figures every performance claim in this course has to be backed by, and learn to read what they are telling you.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

DURATION

Two 2-hour blocks

PREREQUISITE

Labs 1-2, Unit 1, Exercise 01

SUBMIT

lab03-<ROLLNO>.pdf

MARKS

15

OFFERING

Odd Semester 2026-27

The point of this lab

Two figures will appear in every report you write for the rest of this course: a **scaling plot** and a **roofline placement**. This lab is where you learn to produce both properly and, more importantly, to read them.

You will use the functions you wrote in Exercise 01, so bring that file.

BY THE END OF THIS LAB YOU WILL BE ABLE TO

- Measure your machine's peak arithmetic rate and achievable bandwidth, rather than quoting a datasheet.
- Run a strong-scaling study that another student could reproduce.
- Compute efficiency and the Karp-Flatt metric and say what each tells you.
- Distinguish a fixed serial section from overhead that grows with processor count, from the data alone.
- Place a kernel on a roofline and decide whether to keep optimising.

Part 1: Characterise the machine (4 marks)

You cannot draw a roofline without two numbers, and neither should come from a web page.

1.1 Achievable bandwidth

Write a STREAM-style triad: `a[i] = b[i] + s * c[i]` over arrays far larger than L3, timed properly (warm-up, five runs, minimum).

Bytes moved per element: two reads and one write, and on most machines the write also causes a read of the line it lands in. State which assumption you use.

Report GB/s, and compare with the theoretical figure from `lscpu` / the memory specification. You should see 75 to 85 percent.

1.2 Peak arithmetic rate

$$P_{\text{peak}} = \text{cores} \times \text{clock} \times \text{FLOP per cycle per core}$$

Find the FLOP-per-cycle figure for your processor: it depends on the vector width and on whether fused multiply-add counts as two. State your reasoning.

Then **measure** an approachable upper bound: a small loop with many independent fused multiply-adds on data that fits in L1, so nothing is memory bound. You will not reach the theoretical peak; report what you do reach and the ratio.

1.3 The ridge point

Use `ridge_point()` from Exercise 01. Report it in FLOP per byte.

ON ASACOMPUTE · DO THIS ON A COMPUTE NODE, NOT THE LOGIN NODE

`bash srun --cpus-per-task=16 --mem=32G --time=00:15:00 --pty bash` Bandwidth measured on a login node shared with twenty other students is not a measurement of anything.

Part 2: A strong-scaling study (5 marks)

Take the 5-point Jacobi stencil from Assignment 1, or your Exercise 07 workload.

2.1 The protocol

1. Time the **best serial implementation**. This is T_1 . Not your parallel code at one thread.
2. Run at $N = 1, 2, 4, 8, 16, 32$ threads.
3. At each N : one warm-up, five timed runs, keep the minimum, record the spread.
4. Pin threads: `OMP_PROC_BIND=close`, `OMP_PLACES=cores`.
5. Request the **same** allocation at every point, for example `--cpus-per-task=32` throughout, so the one-thread run is not sharing a node.

Use the array-job pattern from Lab 2.

2.2 The numbers

Tabulate, using your Exercise 01 functions:

N	T_N (MIN)	SPREAD	S(N)	E(N)	E (KARP-FLATT)
---	-------------	--------	------	------	----------------

2.3 The plots

Plot A: speedup against N , with the ideal $S = N$ line. **Plot B:** efficiency against N .

Label the axes. Include units. A plot without them earns nothing.

2.4 The reading (this is where the marks are)

Answer:

- At what N does efficiency fall below 0.5? What does that mean for how many cores you should actually request for this problem?
- Is your Karp-Flatt e roughly **constant** or does it **rise** with N ?

DEFINITION · THE SINGLE MOST USEFUL THING IN THIS LAB

A **constant** e means you really do have a fixed serial section of that size, and Amdahl's ceiling is $1/e$. Fixing it means finding and parallelising that section.

A **rising** e means the loss is not a serial section at all: it is overhead that grows with processor count, which Amdahl's model does not describe. Suspect communication, synchronisation, memory bandwidth contention, or load imbalance appearing only at higher thread counts.

You cannot tell these apart from the speedup plot. That is why e is worth computing.

Part 3: Weak scaling (2 marks)

Now fix the work **per thread** and grow the problem with N .

3.1 Run at $N = 1, 2, 4, 8, 16$ with problem size scaled so each thread has a constant share. Plot runtime against N . Perfect weak scaling is a flat line.

3.2 Your line will not be flat. Give two reasons, and say which of them you could test with an additional measurement.

Part 4: The roofline (4 marks)

4.1 For your kernel, compute arithmetic intensity by hand. State your assumption about cache reuse and give the bracketing calculation from Assignment 1 if you have it.

4.2 Draw the roofline for your machine using Part 1's two numbers: a sloped section of gradient β and a flat section at P_{peak} , meeting at the ridge.

4.3 Plot your measured performance at $N = 1$ and at your best N . Use `roofline_fraction()` to report what percentage of **attainable** you reached at each.

4.4 Answer, with a number: is further optimisation worth doing? If yes, name the specific change and the expected gain. If no, say what would have to change before it became worthwhile.

PITFALL · THE MISTAKE THIS PART EXISTS TO PREVENT

Reporting “we reached 2 percent of peak” as though it were a failure. For a kernel whose intensity puts it far to the left of the ridge, 2 percent of peak may be 95 percent of what the machine can deliver. Always report against **attainable**, and show the roofline that makes the number meaningful.

What to submit

lab03 - <ROLLNO> .pdf :

1. Part 1: your two measured machine numbers, with method and comparison to theory, and the ridge point.
2. Part 2: the table, both plots, and your answers to 2.4.
3. Part 3: the weak-scaling plot and your two reasons.
4. Part 4: the roofline with your points marked, and your stopping argument.
5. A reproducibility statement: machine, compiler and version, flags, problem sizes, repetitions, statistic reported.

All source as text, in an appendix.

MARKS	FOR
4	Part 1: measured, justified, with method stated
5	Part 2: correct protocol, correct metrics, and a genuine reading of e
2	Part 3: weak scaling measured and explained
4	Part 4: roofline placement and a defensible stopping argument

NOTE · REUSE THIS

The protocol in Part 2 is the one your mini project must follow. Getting it right here, on a small problem, is much cheaper than getting it wrong in week 15 on a large one.